

MediFlow

Integrantes

- Ayleen Cisternas
- Rafael Cuevas
- Sebastián Esparza

Asignatura

- Taller aplicado de programación

Fecha:

- 26/05/2026

Introducción

En la actualidad, muchos centros médicos y consultorios continúan utilizando procesos administrativos manuales o sistemas fragmentados que dificultan la gestión eficiente de pacientes, citas médicas y comunicación interna. Esta situación provoca retrasos en la atención, errores administrativos, problemas en el control de las citas y una experiencia poco satisfactoria para los pacientes. Frente a esta problemática surge *MediFlow*, una plataforma tecnológica orientada a modernizar y optimizar la gestión clínica mediante herramientas digitales e inteligencia artificial.

MediFlow, la medicina de tu consultorio, busca automatizar procesos críticos dentro de los centros de salud para mejorar tanto la eficiencia operativa como la calidad del servicio entregado a los pacientes. La plataforma integra funcionalidades como agendamiento inteligente de citas, gestión de pacientes, teleconsultas, chatbot de asistencia automatizada, comunicación en tiempo real y administración de información médica, todo dentro de un entorno digital centralizado.

El desarrollo de una solución tecnológica aplicada al área de la salud requiere mantener un equilibrio entre distintos factores fundamentales, tales como seguridad de la información, estabilidad del sistema, escalabilidad y facilidad de uso. Por esta razón, MediFlow fue diseñado bajo una arquitectura moderna y escalable, utilizando tecnologías web actualizadas y metodologías ágiles que permitieran adaptar el sistema a los requerimientos surgidos durante el proceso de desarrollo.

El presente informe tiene como objetivo documentar los principales aspectos técnicos y metodológicos utilizados durante la construcción de la plataforma. A lo largo del documento se describen las metodologías de trabajo implementadas, los patrones de diseño utilizados, la arquitectura del sistema, las tecnologías seleccionadas, la configuración del servidor y los mecanismos de integración entre los distintos componentes del software. Asimismo, se detallan las funcionalidades principales de la plataforma, enfocadas en optimizar la gestión médica, reducir tiempos de espera y mejorar la experiencia tanto de pacientes como del personal de salud.



Índice

Detalle de metodología de desarrollo.....	3
Patrones de diseño.....	4
Arquitectura del sistema.....	5
Frontend.....	6
Backend.....	6
Base de datos.....	6
Lenguajes de programación.....	7
Inteligencia Artificial:.....	7
Detalles de la configuración de servidor.....	8
Aspectos de integración necesarios.....	9
Resumen Integración Frontend–Backend.....	9
Integración Base de Datos.....	10
• Integración Teleconsulta.....	10
Mockup.....	10

Detalle de metodología de desarrollo

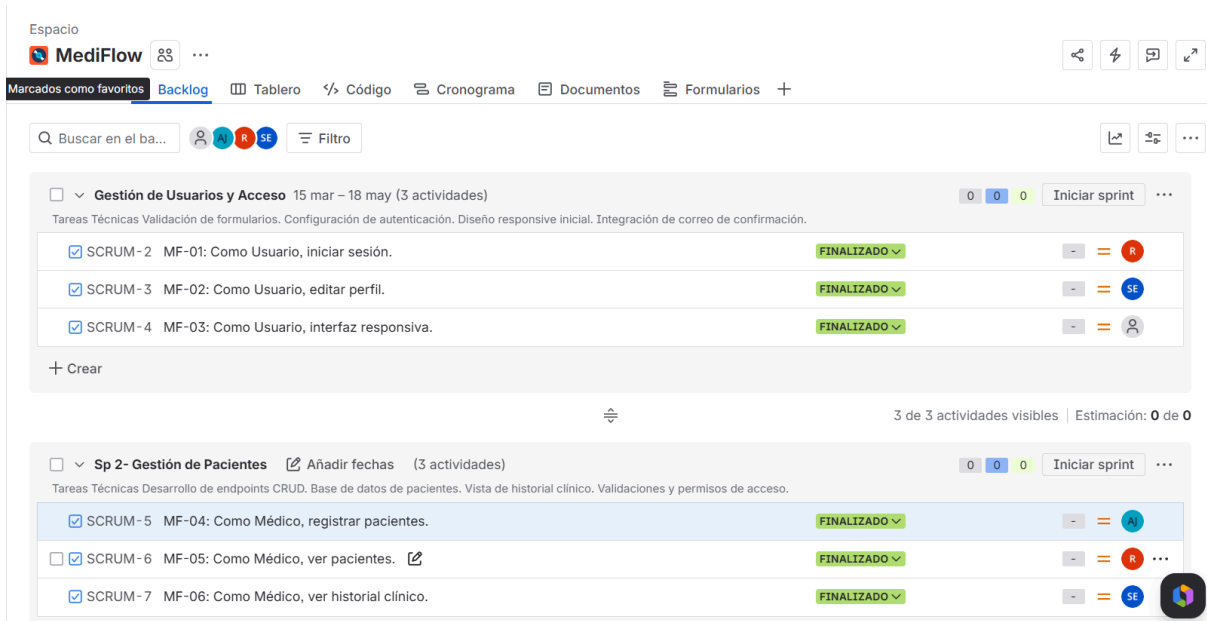
Para el desarrollo del proyecto MediFlow se utilizó la metodología ágil Scrum, debido a que permite trabajar de manera iterativa e incremental, facilitando la adaptación a cambios y la mejora continua del sistema durante todo el proceso de construcción. Esta metodología fue seleccionada porque resulta especialmente adecuada para proyectos tecnológicos, donde los requerimientos pueden evolucionar constantemente y donde es necesario validar funcionalidades de manera progresiva.

Scrum permitió dividir el desarrollo en pequeñas etapas llamadas sprints, cada una con una duración aproximada de una semana. El proyecto fue organizado desde el Sprint 0 hasta el Sprint 15, considerando un total de 16 ciclos de trabajo. Durante las primeras etapas se realizaron actividades de planificación general, organización del equipo, levantamiento de requerimientos, definición de historias de usuario y priorización del backlog del sistema. Posteriormente, el trabajo se enfocó en el desarrollo de funcionalidades específicas del backend y frontend, integración de módulos, implementación de herramientas de inteligencia artificial, pruebas del sistema y despliegue final.

La metodología también permitió mantener una comunicación constante entre los integrantes del equipo, facilitando la identificación temprana de errores y la mejora continua de las funcionalidades implementadas. Gracias a este enfoque iterativo, cada sprint entregó avances funcionales que podían ser revisados, probados y ajustados según las necesidades del proyecto.

Para la administración y seguimiento del desarrollo se utilizó la plataforma Jira, herramienta que permitió organizar las tareas, administrar historias de usuario y monitorear el progreso de cada sprint. A través de Jira fue posible asignar responsabilidades a los integrantes del equipo, gestionar prioridades y visualizar de manera clara el estado de avance de las funcionalidades desarrolladas, tales como el inicio de sesión, gestión de pacientes, administración de citas médicas, historial clínico y teleconsultas.

En su conjunto, el uso de Scrum y Jira permitió mantener una planificación estructurada, mejorar la colaboración entre los miembros del equipo y asegurar el cumplimiento de los objetivos planteados para el desarrollo de MediFlow. Además, tanto la metodología como la plataforma sirvieron para poder mantener organizadas las tareas y comprender el estado de ellas.



Patrones de diseño

Durante el desarrollo de MediFlow se utilizaron distintos patrones de diseño y arquitecturas de software que permitieron mantener una estructura organizada, escalable y fácil de mantener. Entre ellos, se implementó el patrón de arquitectura MVC (*Model View Controller*), utilizada principalmente en el backend desarrollado con Spring Boot, lo que permitió separar las responsabilidades del sistema en diferentes capas, facilitando el mantenimiento y la escalabilidad del proyecto.

Model:

- Usuario
- ProfesionalSalud
- Cita
- Especialidad

Controller:

- UsuarioController
- CitaController
- EspecialidadController
- CentroSaludController

View:

- ReactJS

Beneficios:

- Separación de responsabilidades.
- Mantenimiento sencillo.
- Mejor escalabilidad.

La implementación de MVC aportó beneficios importantes al proyecto, tales como una clara separación de responsabilidades, mayor facilidad de mantenimiento del código y una arquitectura más escalable para futuras mejoras.

Además, se implementó el patrón DTO (Data Transfer Object), utilizando clases como *UsuarioDTO* para transferir información entre backend y frontend de manera segura y eficiente. Este patrón permitió evitar la exposición directa de las entidades del sistema y mejorar el control sobre los datos enviados al cliente.

Arquitectura del sistema

MediFlow fue desarrollado bajo una arquitectura Cliente–Servidor de tres capas, permitiendo separar adecuadamente la presentación visual, la lógica de negocio y el almacenamiento de datos. Esta estructura facilita la escalabilidad del sistema, mejora el mantenimiento y permite integrar nuevas funcionalidades de manera más eficiente.

La capa Frontend fue desarrollada utilizando ReactJS junto con Vite, React Router DOM, HTML y CSS. Esta capa es la encargada de proporcionar las interfaces visuales del sistema, permitiendo a los usuarios interactuar mediante formularios, paneles administrativos, módulos de gestión médica y herramientas de teleconsulta. Además, el frontend consume las API REST del backend y administra la navegación dinámica dentro de la aplicación.

Por otra parte, la capa de lógica de negocio o Backend fue desarrollada utilizando Spring Boot y Java 17. Esta capa es responsable de procesar las solicitudes realizadas por el frontend, aplicar las reglas de negocio del sistema, validar información, administrar autenticación y gestionar la comunicación. En el caso de las teleconsultas, la comunicación en tiempo real se realiza mediante tecnologías WebRTC y WebSocket, permitiendo la transmisión de audio, video y mensajería instantánea entre pacientes y profesionales de la salud.

Además, el backend integra un módulo de inteligencia artificial conectado a Ollama 3.2, encargado de procesar consultas enviadas por los usuarios desde el chatbot del sistema. Este módulo permite generar respuestas automáticas utilizando procesamiento de lenguaje natural, entregando asistencia básica y orientación inicial a pacientes.

La capa de datos utiliza MySQL como sistema de gestión de base de datos. En esta capa se almacena información relacionada con pacientes, profesionales de la salud, citas médicas, especialidades, historiales clínicos y registros asociados a teleconsultas y chatbot.

El flujo principal de funcionamiento del sistema sigue la siguiente estructura:

Usuario → Frontend React → API REST → Backend Spring Boot → Base de Datos MySQL

El flujo de IA comprende lo siguiente:

Usuario → Chat React → API REST → Spring Boot → Ollama 3.2 → Respuesta IA

En resumen, MediFlow utiliza arquitectura Cliente–Servidor de tres capas:

Frontend

Tecnología

- React
- Vite
- React Router
- CSS

Funciones

- Interfaces visuales.
- Formularios.
- Navegación.
- Consumo APIs.
- Videollamada.

Backend

Tecnología

- Spring Boot
- Java
- Hibernate
- Spring WebSocket

Funciones

- Reglas de negocio.
- Validación.
- API REST.
- Autenticación.
- Comunicación en tiempo real.

Base de datos

Tecnología

- MySQL

Entidades

- Usuarios
- Profesionales
- Especialidades

- Citas
- Centros de salud

Flujo: Usuario → React → API REST → Spring Boot → MySQL

Teleconsulta: Usuario → WebRTC → WebSocket → Profesional

Lenguajes de programación

Backend

- Java 17
- Spring Boot 4
- Hibernate ORM
- Spring Data JPA
- Maven

Frontend

- ReactJS
- Vite
- Javascript
- HTML
- CSS
- React Router DOM

Base de datos

- MySQL

Comunicación tiempo real

- WebSocket
- SockJS
- WebRTC

Herramientas

- Visual Studio Code
- Postman
- GitHub

Inteligencia Artificial:

- Ollama 3.2
- Modelo LLM local

- Procesamiento de lenguaje natural
- Integración API Backend

Detalles de la configuración de servidor

El backend del sistema fue configurado utilizando un servidor embebido Apache Tomcat 11, ejecutándose sobre el puerto 8080. La configuración incluyó distintas dependencias necesarias para el funcionamiento de la plataforma, entre ellas Spring Web, Spring Data JPA, Spring WebSocket, MySQL Driver, Validation y DevTools.

Para permitir la comunicación entre frontend y backend se implementó configuración CORS (*Cross-Origin Resource Sharing*), habilitando las conexiones entre el frontend ejecutado en <http://localhost:5173> y el backend desplegado en <http://localhost:8080>.

En cuanto a las funcionalidades de comunicación en tiempo real, se configuró un endpoint WebSocket ubicado en /ws/video, utilizando STOMP como protocolo de mensajería y /topic como broker de distribución de mensajes. Esto permitió gestionar videollamadas, señalización WebRTC e intercambio de mensajes en tiempo real entre usuarios.

Para el funcionamiento del chatbot inteligente fue necesaria la ejecución local del servicio Ollama 3.2, el cual se integró con el backend mediante solicitudes HTTP. Esto permitió procesar consultas de usuarios y generar respuestas automáticas directamente desde el servidor de aplicación.

Configuración backend

- Puerto: 8080
- Servidor embebido: Tomcat 11

Dependencias utilizadas

- Spring Web
- Spring Data JPA
- Spring WebSocket
- MySQL Driver
- Validation
- DevTools

Configuración CORS

Se implementó configuración CrossOrigin para permitir comunicación entre:

- Frontend: <http://localhost:5173>
- Backend: <http://localhost:8080>

WebSocket

- Endpoint: /ws/video
- Broker: /topic
- Aplicación: /app

Configuración STOMP para transmisión de mensajes.

Aspectos de integración necesarios

Uno de los aspectos más importantes del proyecto fue la integración entre los distintos componentes tecnológicos utilizados en MediFlow AI.

La integración entre frontend y backend se realizó mediante API REST utilizando solicitudes HTTP y transferencia de información en formato JSON. A través de esta comunicación se implementaron funcionalidades como registro de usuarios, autenticación, creación de citas médicas, gestión de pacientes y acceso a información clínica.

La integración del chatbot inteligente se realizó conectando el frontend desarrollado en React con el backend en Spring Boot, el cual actúa como intermediario entre la interfaz de usuario y el servicio Ollama 3.2. Las consultas realizadas por los usuarios son enviadas al modelo de lenguaje, procesadas y posteriormente retornadas al sistema como respuestas automatizadas.

Resumen Integración Frontend–Backend

Comunicación mediante API REST utilizando Fetch

Funciones implementadas

- Registro
- Login
- Crear citas
- Obtener usuarios
- Videollamada

Formato: JSON

- { correo:"usuario@correo.cl", password:"123456" }

Integración Base de Datos

- Utilización de Spring Data JPA.
- Mapeo mediante entidades Hibernate.
- Integración Teleconsulta

WebRTC:

Permite transmisión de:

- Audio
- Video
- Pantalla

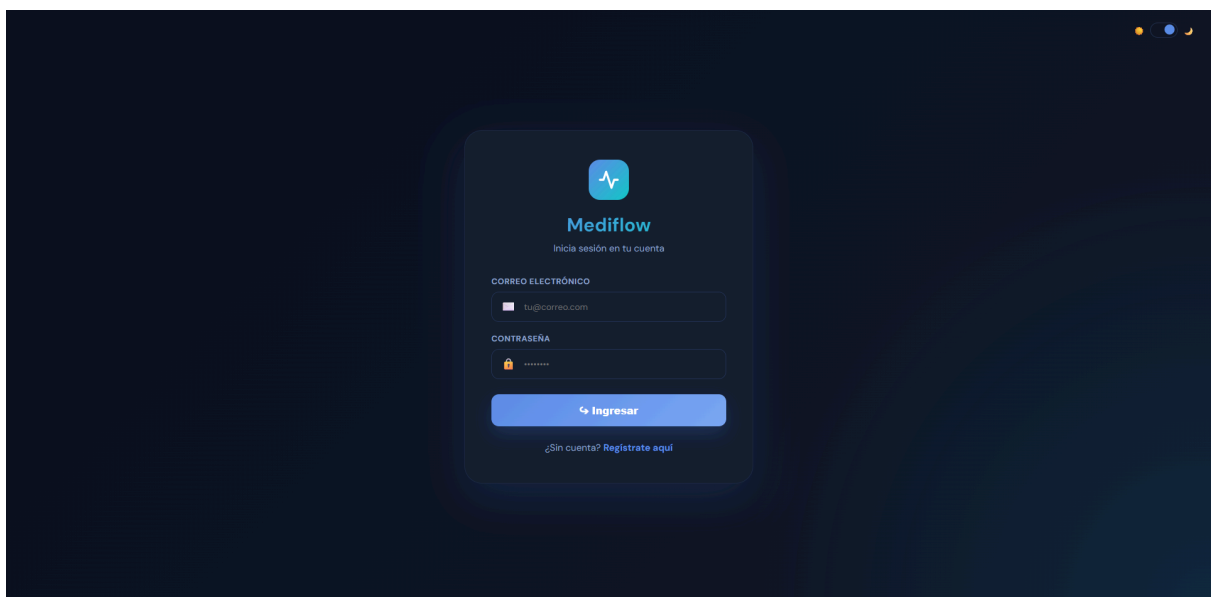
WebSocket:

Permite:

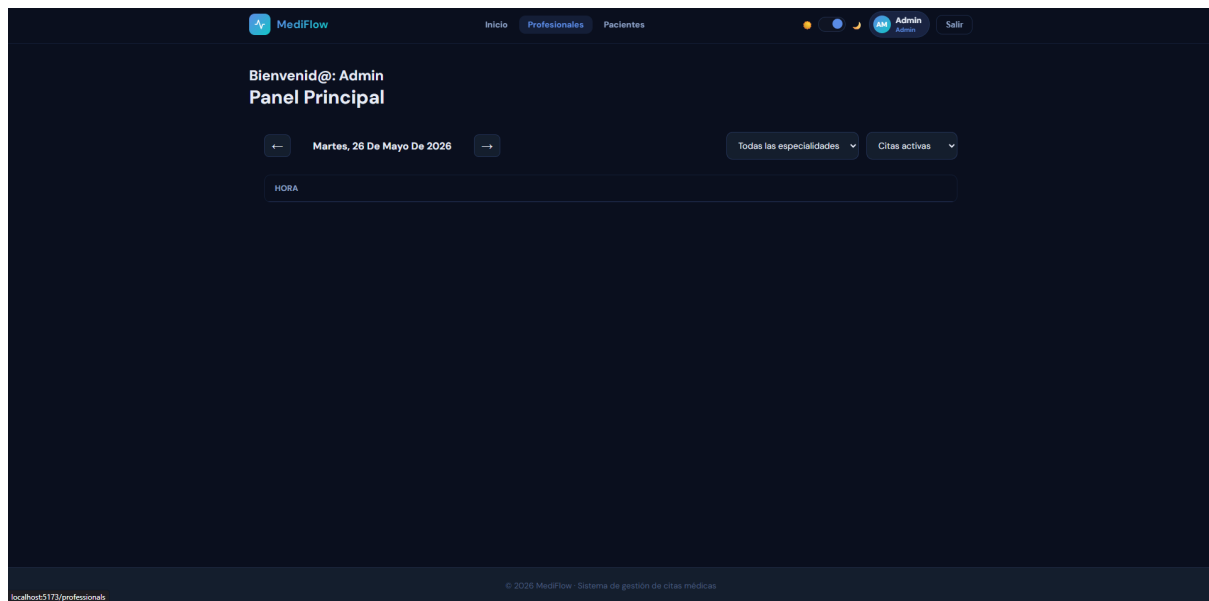
- Señalización
- Intercambio SDP
- ICE candidates
- Chat

Mockup

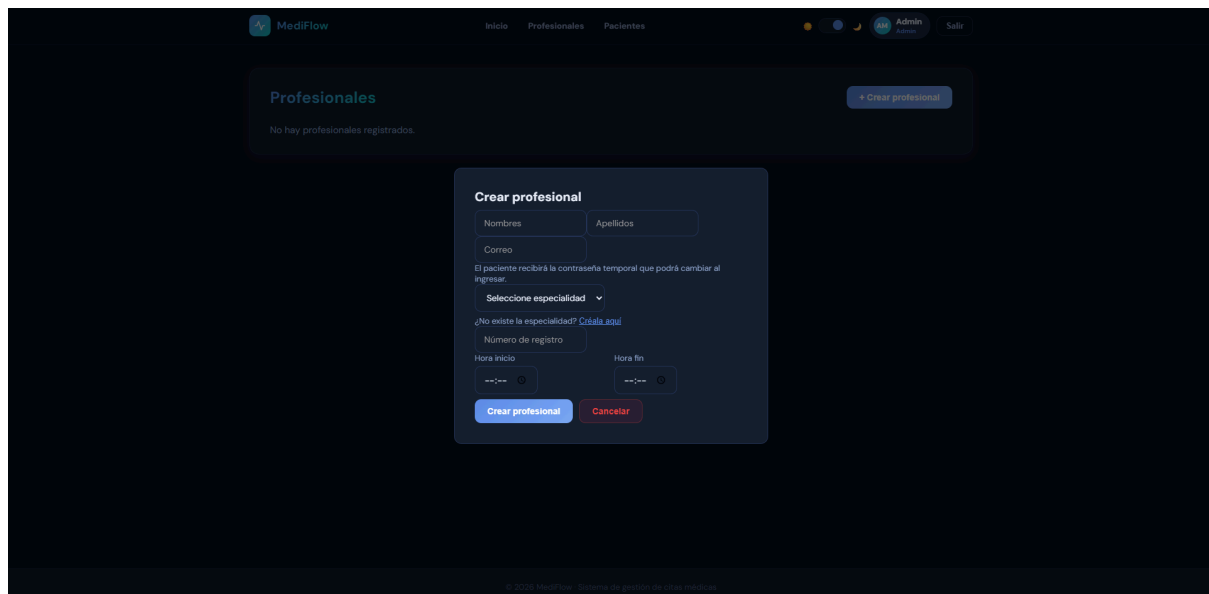
Login



Home de admin



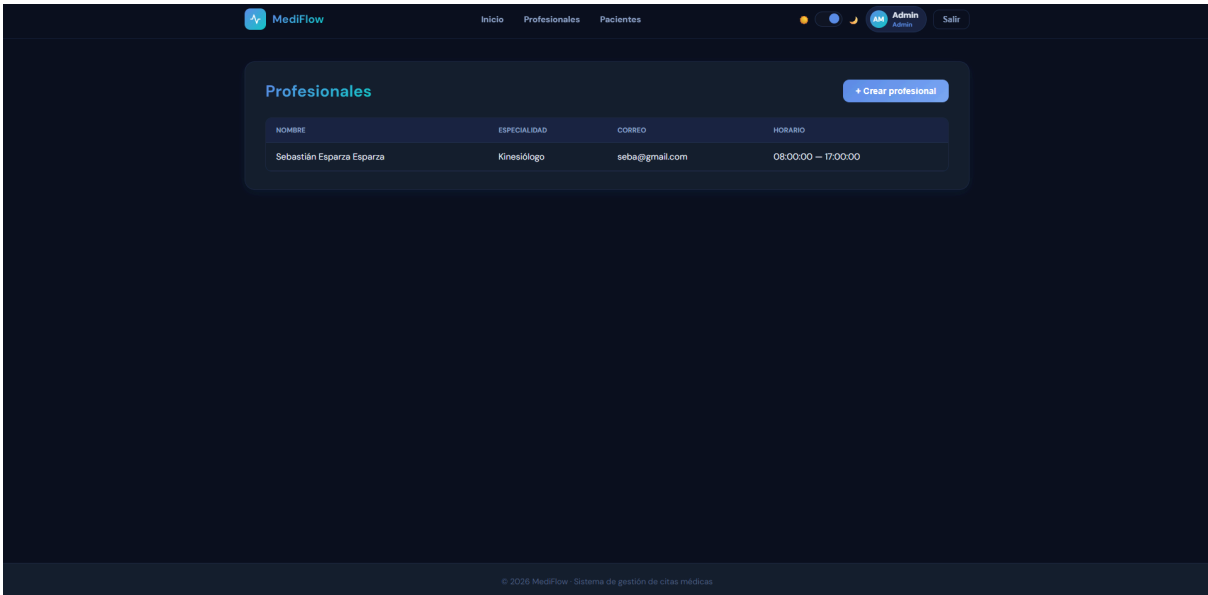
Crear profesional desde admin



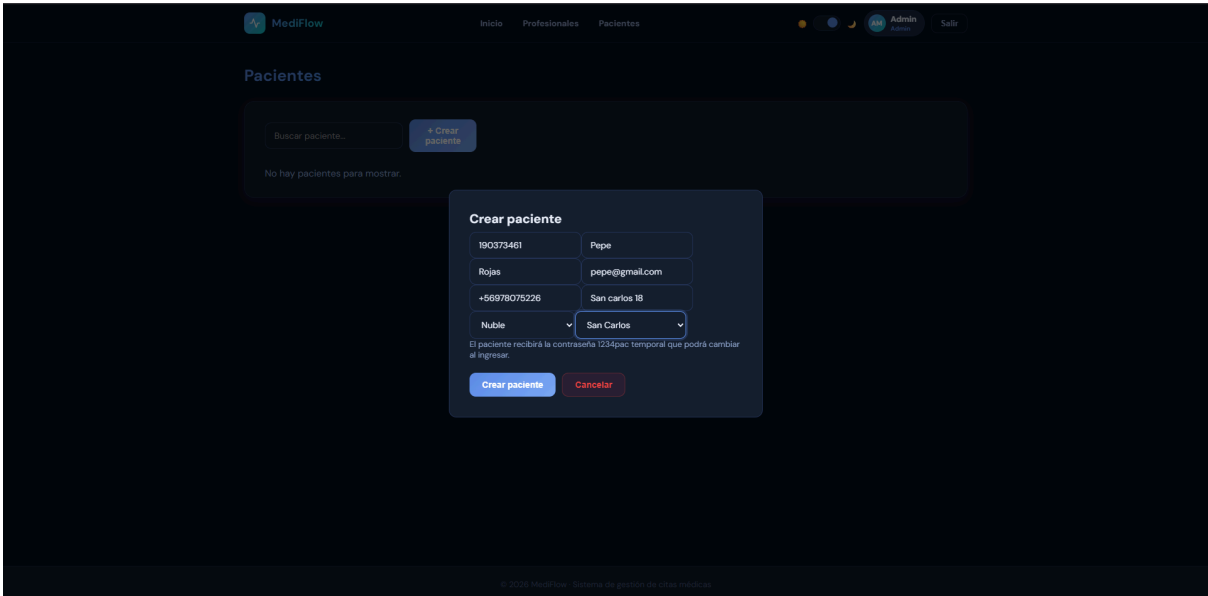
Crear especialidad para profesional



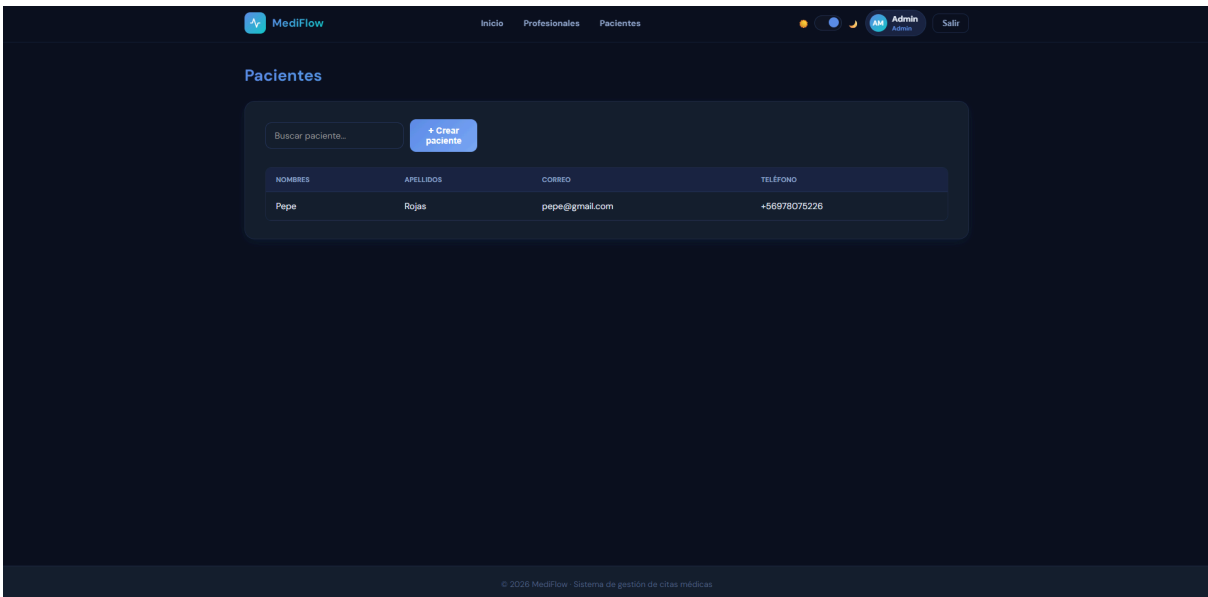
Lista de profesionales



Crear paciente desde admin



Lista de pacientes desde admin



Cambiar contraseña temporal.

Cambiar contraseña

Nueva contraseña

Confirmar contraseña

Guardar contraseña

Home de profesional

MediFlow

Inicio

Mis citas

Pacientes

Historial clínico

Sebastián Esparza

Profesional

Salir

Bienvenid@: Sebastián Esparza

Panel Principal

←

Martes, 26 De Mayo De 2026

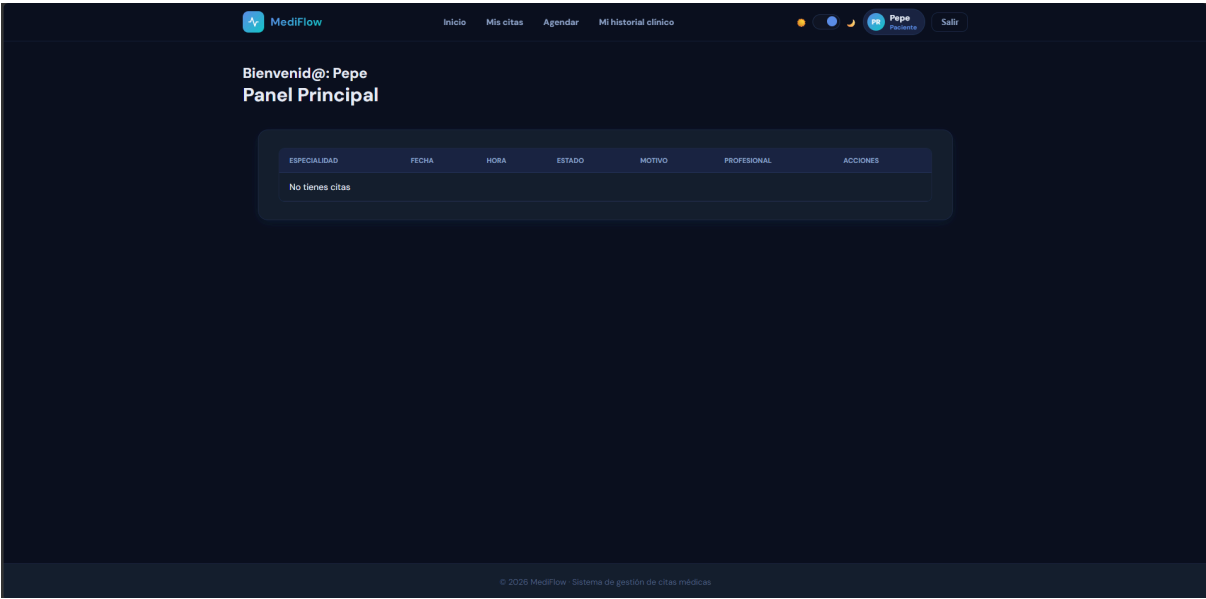
→

Citas activas

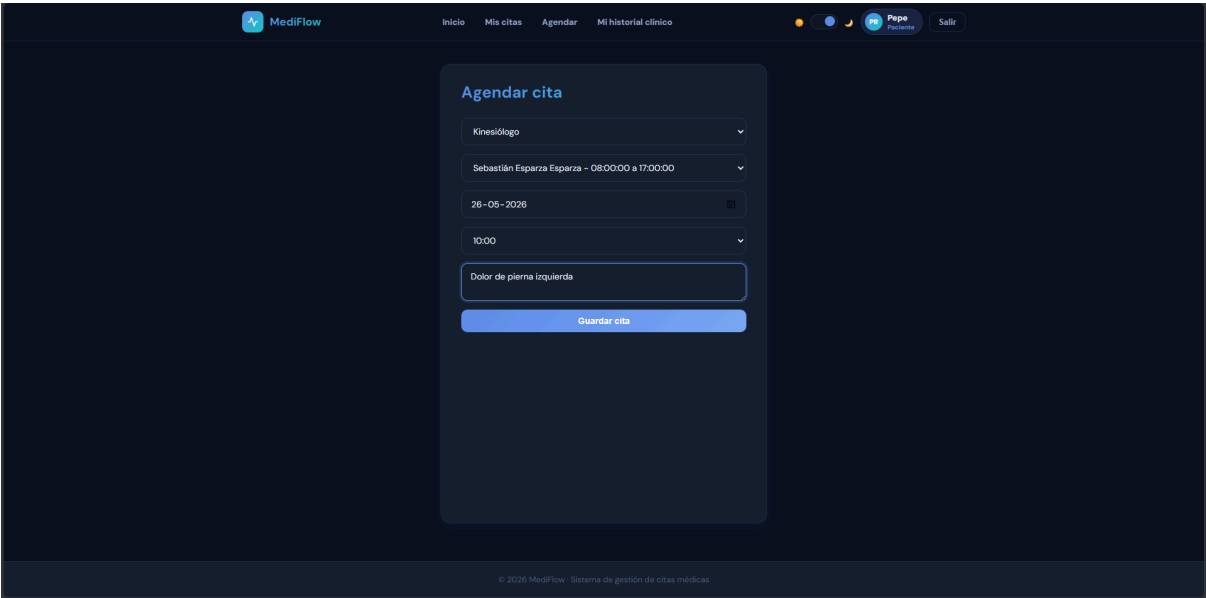
▼

HORA	SEBASTIÁN ESPARZA ESPARZA KINESIOLOGO
08:00	
08:30	
09:00	
09:30	
10:00	
10:30	
11:00	
11:30	
12:00	
12:30	
13:00	
13:30	

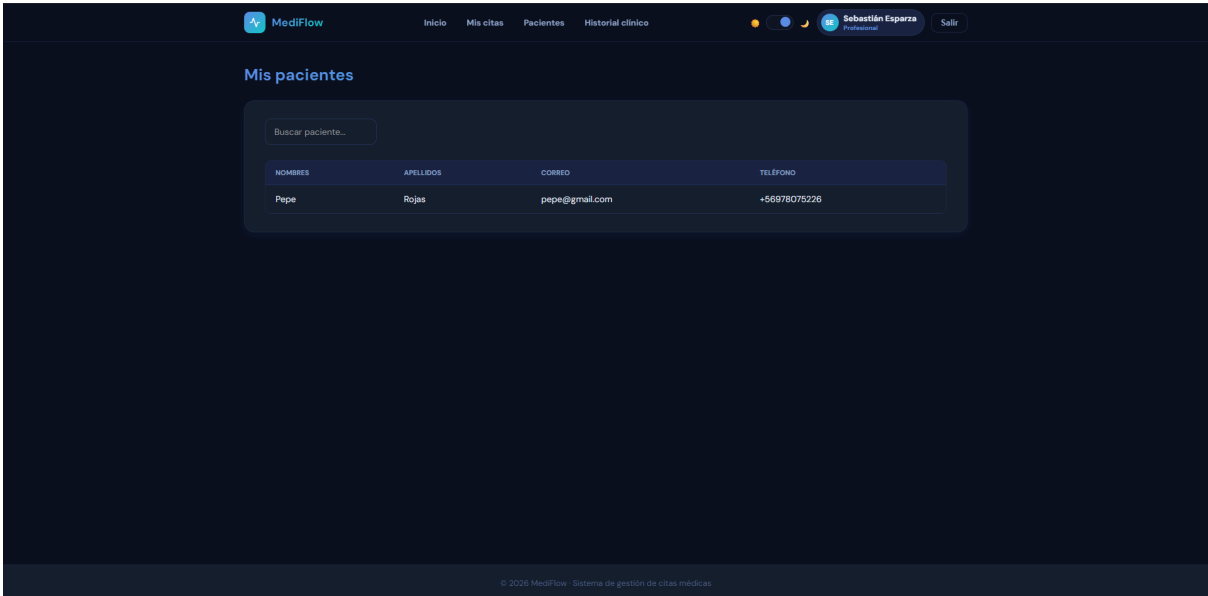
Home de paciente



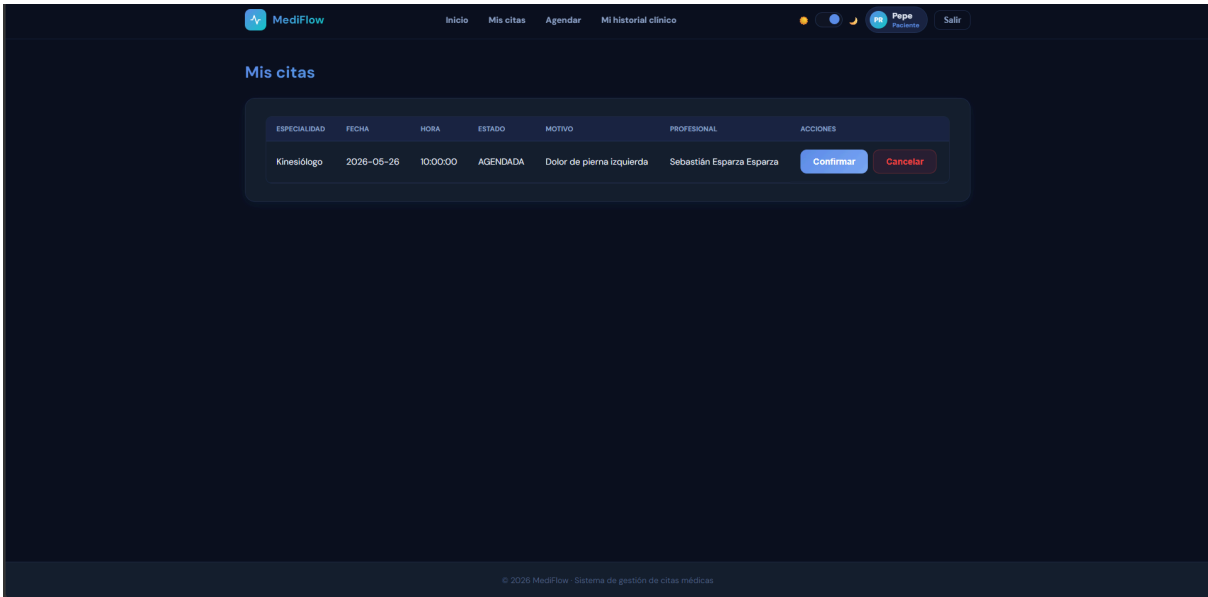
Vista de agendar cita



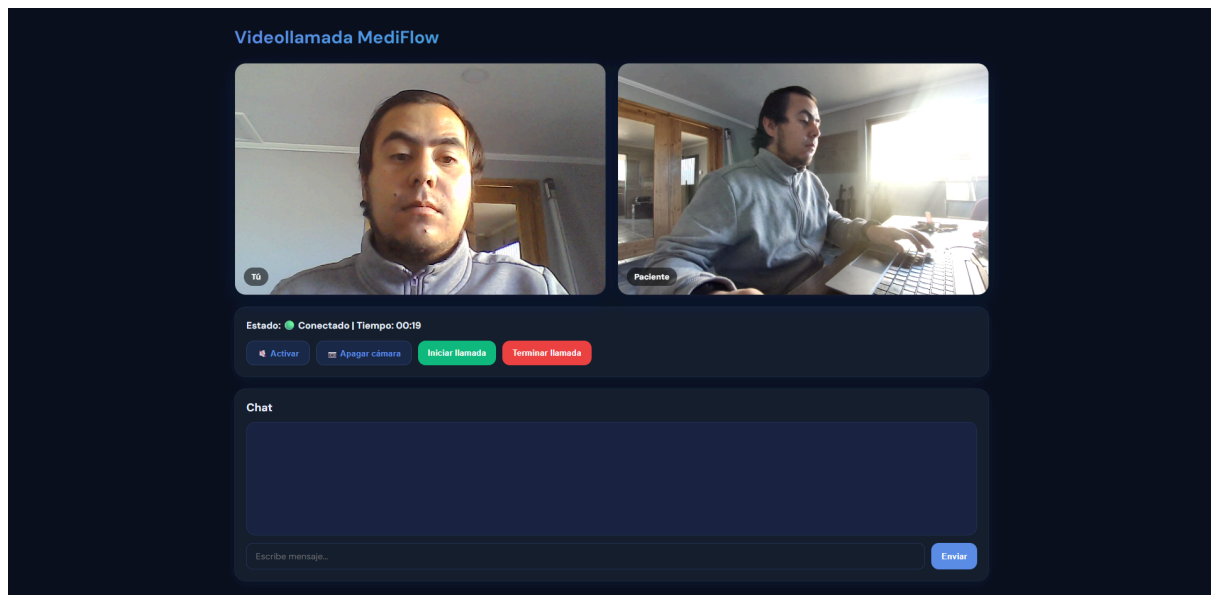
Lista de pacientes desde profesional



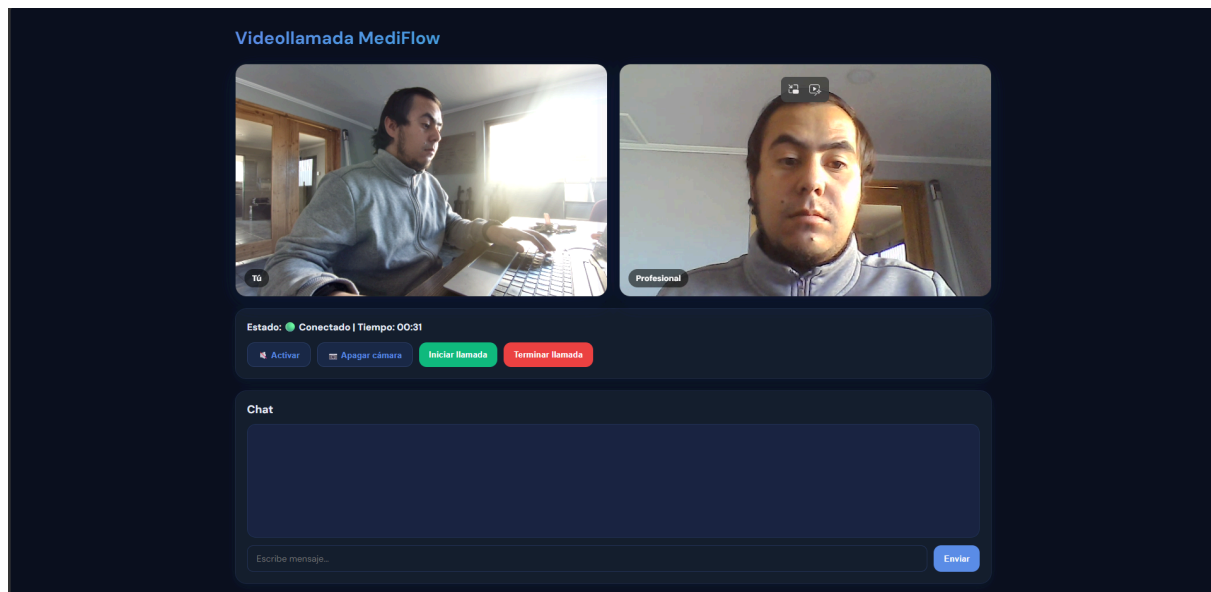
Vista de citas del paciente



Videollamada vista del profesional



Videollamada vista del paciente



Ficha clinica vista del profesional posterior a finalizar cita

Ficha clinica

Diagnóstico

Fractura de rodilla izquierda

Tratamiento

Cirugía
Reposo
Rehabilitación

Observaciones

Acudir a urgencia en caso de mucho dolor

Guardar historial y finalizar

Volver

© 2025 MediFlow - Sistema de gestión de citas médicas

Historial clínico vista del paciente

MediFlow

InicioMis citasAgendarMi historial clínico

Pepe PacienteSalir

Mi historial clínico

PACIENTE	PROFESIONAL	ESPECIALIDAD	FECHA	DIAGNÓSTICO	TRATAMIENTO	OBSERVACIONES
Pepe Rojas	Sebastián Esparza Esparza	Kinesiólogo	2028-05-26T09:45:10	Fractura de rodilla izquierda	Cirugía Reposo Rehabilitación	Acudir a urgencia en caso de mucho dolor

© 2025 MediFlow - Sistema de gestión de citas médicas

Historial clinico vista del profesional

MediFlow

[Inicio](#)[Mis citas](#)[Pacientes](#)[Historial clínico](#)

Sebastián Esparza

Profesional

Salir

Historial clínico

PACIENTE	PROFESIONAL	ESPECIALIDAD	FECHA	DIAGNÓSTICO	TRATAMIENTO	OBSERVACIONES
Pepe Rojas	Sebastián Esparza Esparza	Kinesiólogo	2020-05-20T09:45:10	Fractura de rodilla izquierda	Cirugía Reposo Rehabilitación	Acudir a urgencia en caso de mucho dolor

© 2020 MediFlow - Sistema de gestión de citas médicas

Chatbot cerrado

Todas las especialidades

Citas activas

PRUEBA KKKK
ESPECIALIDAD 1

ALON DRA
ESPECIALIDAD 1

Asiste

Chatbot conversación

